Neža Podpeskar

AI WORKFLOW REFLECTION

Throughout this project, I used AI tools in different ways depending on the task. I mainly used Claude in chat for step-by-step reasoning, debugging, explanations, and checking ideas. I used Claude Code in the terminal for larger implementation tasks when building and refining the Streamlit app. Alongside that, I also used ChatGPT for design ideas, UI styling prompts, layout improvements, and improving the overall visual appearance of the application.

From the beginning, I tried to be careful about how I used AI. I intentionally started the project inside Jupyter Notebook instead of immediately generating an app, because I wanted to understand the preprocessing, leakage prevention, feature engineering, and model evaluation myself (lectures about Machine Learning Parts 1, 2 & 3 helped me a lot!!). I learned much more that way than I would have by skipping directly to generated code.
One important thing I learned was that AI-generated outputs still need verification. During deployment, I noticed that some graphs in the Market Insights section looked different from my notebook analysis. After checking the issue, I realized Claude Code had accidentally built some visualizations from the raw Excel dataset instead of using the cleaned data pipeline from my notebook. Initially, Claude suggested temporarily leaving it as it was because fixing it properly would take more time. I decided against that and rebuilt it to use the same cleaned and verified data used during training and analysis.

My general approach to verification was basically to check everything twice. I compared notebook outputs with app outputs, checked preprocessing logic carefully, and tried not to trust generated code automatically just because it looked correct. That was probably one of the biggest lessons from the project.

I also considered using MCP servers during development. I had previously experimented with a fetch-based MCP server in another assignment and initially thought about using one here as well. In the end, I decided not to use MCP servers in the final implementation because I wanted the deployed app to remain as stable and reliable as possible without introducing additional complexity (eg. all the pictures in my web app are downloaded and stored in my directory, not directly online).

In terms of effort, the project took quite a lot of time. Most of that time was spent inside Jupyter Notebook working on data analysis, debugging, preprocessing, and validating the ML pipeline rather than building the app interface itself. Deployment, styling, and feature polishing also took some time because I wanted the final application to feel polished and professional rather than looking like a default dashboard.
Overall, I learned an enormous amount from this project, not only technical skills, but also that when something breaks, it is not the end of the world. That there is usually a solution to every problem, and catching errors is part of the development process, not a sign of failure. Over time I became much more comfortable slowing down, checking pipelines carefully, and solving problems step by step instead of panicking when something broke.